Full Stack Assignment – 42

Assignment ID: FSD-42

Title: Dependency-Based Task Execution System

Duration: 3 Hours

1️⃣ Objective

Design and develop a **Dependency-Based Task Execution System** where tasks can depend on the completion of other tasks.

The system must strictly enforce dependency rules and ensure **data persistence using a database**.

2️⃣ User Roles

CREATOR

- Register and log in to the system
- Create tasks with dependencies
- View task execution status

EXECUTOR

- Register and log in to the system
- Execute assigned tasks
- View task dependency status

3️⃣ Functional Requirements

Authentication & Authorization

- Implement user registration and login
- Use **JWT-based authentication**
- Protect all backend APIs using JWT
- Enforce **role-based access control** on APIs and frontend views

Task Management

- CREATOR can create tasks with:
  - Task title
  - Description (optional)
  - Dependency task (optional – single dependency is sufficient)
- All task data must be stored in the database
- Task status must be retrieved dynamically from the database

Dependency Handling

- Tasks may depend on another task
- EXECUTOR can execute a task only if its dependency is completed
- Task execution state must be persisted in the database

4️⃣ Business Rule (Critical)

⚠️ **A task must not be executed unless all its dependent tasks are completed.**

Any attempt to execute a task before its dependency is completed must be rejected at the backend.

5️⃣ Technical Constraints (Mandatory)

- All task and dependency data **must persist in a database**
- Hardcoded or in-memory dependency logic is **not allowed**
- Backend APIs must reflect actual database state
- JWT authentication and role validation are mandatory
- Environment variables must be used for sensitive configuration

6️⃣ Frontend Requirements

- Login and registration pages
- Role-based dashboards for CREATOR and EXECUTOR
- Task creation form with dependency selection
- Task execution view displaying dependency status clearly

UI design is not evaluated; dependency logic and correctness are the priority.

7️⃣ Submission Requirements (Strict)

You must submit **all of the following**:

GitHub Repository

- Backend source code
- Frontend source code
- Proper folder structure
- Clear commit history

Live Deployment

- Live frontend deployment link
- Live backend deployment link
- Backend must be connected to the database

README.md (Mandatory)

The README file **must include**:

- Project overview
- Tech stack used
- User roles and permissions
- API endpoints
- Database schema / collections
- Instructions to run locally
- Live frontend and backend links

8️⃣ Evaluation Note

Evaluation focuses on **dependency enforcement, role separation, database usage, and end-to-end execution**, not on UI polish or extra features.